

3. Bubble Sort with Big-O Analysis

Aim

To write a Python program to implement **Bubble Sort** and analyze its **Big-O time complexity**.

Algorithm

1. Start.
2. Read an array of elements.
3. Compare adjacent elements.
4. If the first element is greater than the second, swap them.
5. Repeat the process for all elements.
6. After each pass, the largest unsorted element moves to the end.
7. Repeat until the array is sorted.
8. Print the sorted array.
9. Stop.

Source Code

```
def bubble_sort(arr):  
  
    n = len(arr)  
  
    for i in range(n):  
  
        for j in range(0, n - i - 1):  
  
            if arr[j] > arr[j + 1]:  
  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr  
  
arr = [5, 3, 8, 4, 2]  
  
print("Sorted Array:", bubble_sort(arr))
```

Output



The screenshot shows a code editor with a file named 'main.py'. The code implements a bubble sort function. The editor has a 'Visualize' button and a 'Run' button. The output panel on the right shows the result of running the code.

```
1 def bubble_sort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         for j in range(0, n - i - 1):
6             if arr[j] > arr[j + 1]:
7                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
8
9     return arr
10
11
12 arr = [5, 3, 8, 4, 2]
13
14 print("Sorted Array:", bubble_sort(arr))
```

Output: Sorted Array: [2, 3, 4, 5, 8]

Big-O Analysis

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Space Complexity: $O(1)$

Result

Thus, the Bubble Sort program was successfully implemented. The worst-case time complexity is **$O(n^2)$** and the space complexity is **$O(1)$** .